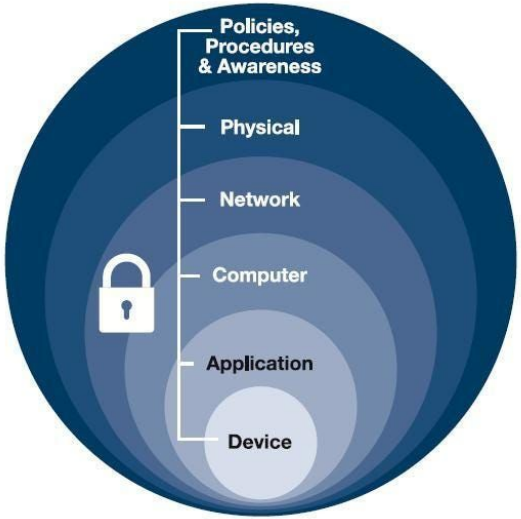


Criptografía y Seguridad

Principios de Diseño
de Seguridad
Vulnerabilidades

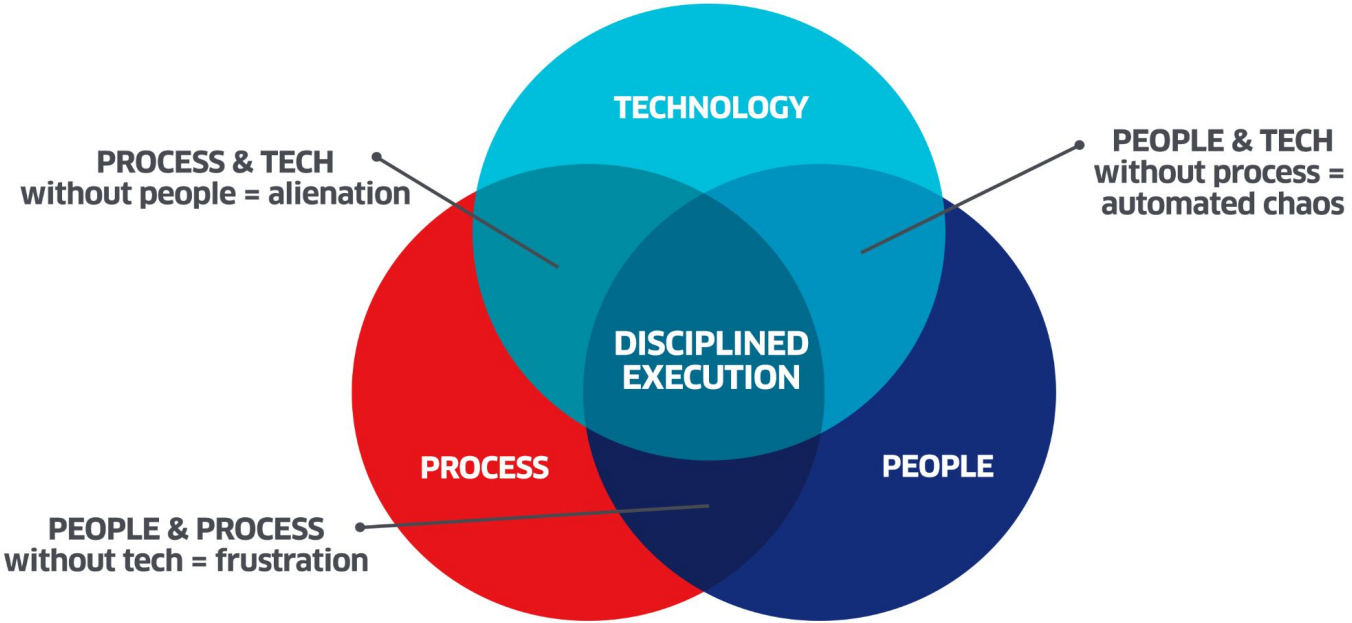
Principios de Diseño

- Bases que promueven un diseño que tenga como resultado un sistema robusto y seguro
- Son principios guía de alto nivel, que deben adaptarse a las necesidades puntuales
- Se basan las recomendaciones de los modelos de seguridad.
- Se deben aplicar en todas las capas en donde funciona nuestro sistema.
- Simples y restrictivos



Principios de Diseño

- No deben perder la perspectiva de cómo se implementan o quiénes los van a implementar y usar.



Principios de Saltzer & Schroeder

Propuestos en 1975 para el diseño de sistemas operativos seguros.

Demostraron aplicabilidad en diseños generales, y sistemas modernos.

Son la base de buenas prácticas modernas y estrategias de gobierno en todas las capas

1. Menor Privilegio (Least privilege)

- Se debe asegurar que los sujetos sólo tengan el acceso necesario para realizar su trabajo.
- Por ejemplo, si diseña un sistema que contiene información financiera confidencial de los clientes, es una buena práctica limitar quién puede acceder a esa información.
  - La persona que responde al teléfono y programa reuniones probablemente no necesite acceder a toda la información confidencial.
  - Por otro lado, un administrador de cuentas probablemente sí necesite acceder a esa información. La clave es asegurarse de que el administrador de cuentas no tenga acceso a la información de cuentas que no administra.

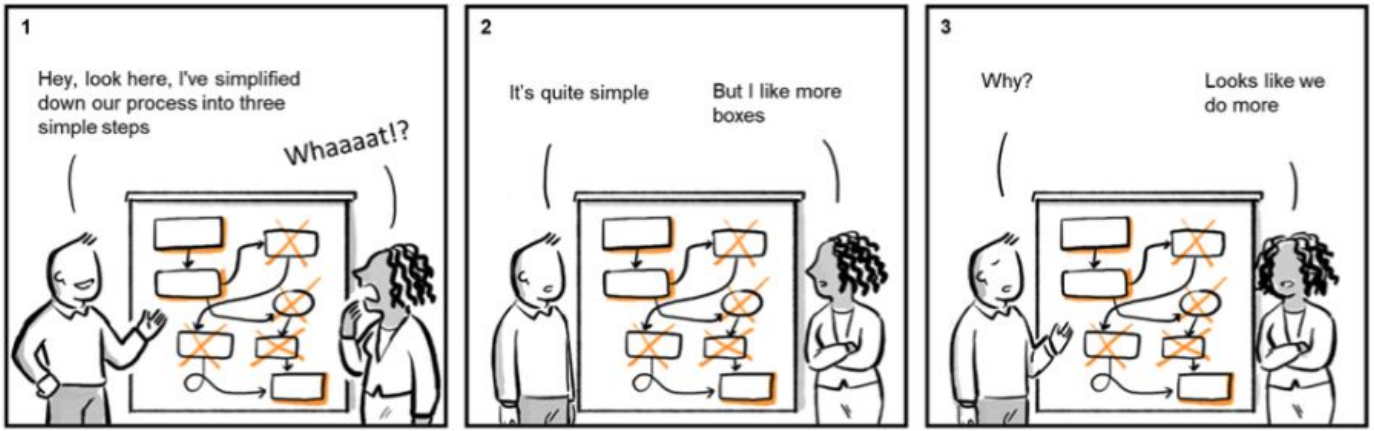
2. Fallar de forma segura (Fail-safe defaults)

- A la hora de revisar, listas de permitidos (whitelists) son mejores que listas de excluidos (blacklists)
- Un sistema no abre puertos innecesariamente
- No hay cuentas de usuario por defecto con una contraseña conocida
- Un sistema que presenta una falla en un proceso o control de seguridad debe moverse a un estado seguro.

- Ejemplo:
  - Si mi sistema de autenticación no se puede conectar a la base de datos para verificar un permiso, debe rechazar el pedido.

3. Simplicidad (Economy of mechanism)

- Se debe evitar el uso de arquitecturas muy sofisticadas al desarrollar controles de seguridad.
- Sistemas muy complejos aumentan el riesgo de errores y la dependencia de otros componentes para aplicar el control.



3. Simplicidad (Economy of mechanism)

El principio busca reducir la superficie de ataque, e incluye la complejidad de datos.
Por ejemplo reducir al mínimo la cantidad de tuplas (sujeto, acceso, objeto).

- Cada uno de esos accesos es susceptible a ser explotado, de modo que al reducir la cantidad de accesos disponibles para un sujeto, reduzco el riesgo de un ataque efectivo.



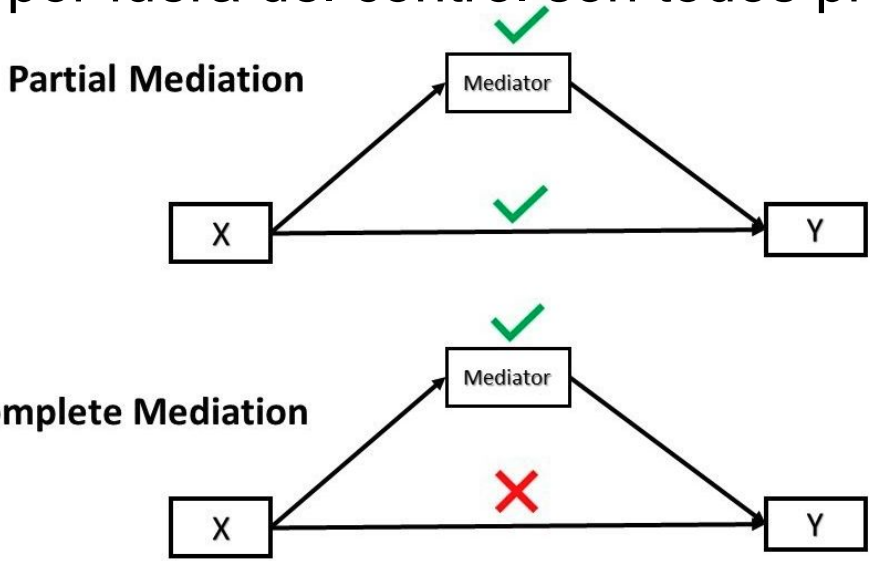
3. Simplicidad (Economy of mechanism)



4. Mediación completa (Complete mediation)

Si hay un control, este debe aplicar a todas las interacciones alcanzadas.

La existencia de muestreo de algunas transacciones, controles al azar, o entidades por fuera del control son todos problemas potenciales.



5. Sistema Abierto (open design)

- El principio de sistemas abierto (Open design) define que no debe basarse la seguridad en mantener su diseño oculto.
- Todo secreto conocido por lo menos una persona, puede ser revelado.
- En los algoritmos de encripción, lo que se guarda es la clave, y se cambia frecuentemente. El algoritmo es público y conocidos por todos.
- El mismo concepto se aplica para los controles de seguridad.



6. Segregación de Tareas (separation of privilege)

- Toda tarea crítica no debe quedar bajo el control de un sólo sujeto. Más de un sujeto debe ser parte del proceso para asegurar que hay un control entre partes.
- Se aplica muchas veces el principio de **control por oposición de intereses**.
  - Las partes que participan en el proceso tienen objetivos opuestos, y en el equilibrio, se evita el abuso.
  - Supongamos un proceso de ventas, donde se pueden dar descuentos.
  - Para aprobar la venta, el vendedor debe pedir la aprobación de un descuento. Le conviene descontar mucho para asegurar la venta.
  - El financiero que aprueba el descuento, quiere descontar lo menos posible, ya que pierde plata en la venta.
  - Hay oposición de intereses entre partes.

7. Mecanismos exclusivos (Defense-in-Depth)

- Evitar que distintos mecanismos compartan recursos, algoritmos y hardware.
- El es principio del modelo de defensa en profundidad (Defense-in-depth).
- Se debe diseñar los controles compensatorios, de mitigación o reacción para ser accionados si otro control de seguridad falla o es superado.
- Ejemplo:
  - Hacemos segregación de tareas y aplicamos mínimos privilegios, pero una vulnerabilidad en el sistema permite a un atacante llevar adelante una venta sin solicitar las respectiva autorizaciones.
    - Qué controles se agregan para monitorear esa situación?
    - Como se pueden crear alertas que bloqueen la transacción.
    - Se puede hacer una confirmación?

8. Menor asombro (psychological acceptability)

- El peor enemigo de cualquier diseño de seguridad es el usuario no alineado.
- Si alguien tiene que elegir entre seguridad y cumplir con sus deberes, va a priorizar lo segundo.
- Los controles debe ser simples de usar, inclusive para una persona sin conocimientos técnicos. No deben complicar el negocio o los procesos del negocio.
- La concientización del problema y el valor de la seguridad es fundamental para lograr la aceptación del control.





Criptografía y Seguridad

Ejercicio del uso de estos principios

Caso Aplicación Web

- Considerar un sistema de ventas que corre en un servidor Linux.
- Posee un Web Server que sirve paginas escritas en PHP, que se conecta con una base de datos Postgres instalada en el mismo servidor.
- La autenticación y gestión de usuarios lo maneja una API interna. Este servicio también está instalados dentro del servidor. También usa la misma base de datos y schema. Tiene tablas dedicadas.
- La base de datos tiene un único schema y un sólo usuario.
- Todos los procesos corren como root.



Criptografía y Seguridad

Vulnerabilidades

Amenazas & Vulnerabilidades

- Cualquier elemento que pueda afectar la confidencialidad, integridad o disponibilidad se los denomina una amenaza (**Threat**).
- **Vulnerabilidad** es una debilidad en un activo o grupo de activos que permite que un atacante pueda aproveche para lograr un acceso que no tiene permitido.
- Una amenaza y una vulnerabilidad generan un **riesgo**.



https://www.bmc.com/blogs/security-vulnerability-vs-threat-vs-risk-whats-difference/

Tipos de Amenazas

- Existe varios criterios para analizar los tipos de amenazas:
  - Según su origen con respecto a la organización:
    - Internos: la amenaza se origina dentro de la organización
      - un empleado que hace fraude,
      - o una falla en un equipo que deja sin servicios
    - Externos: la amenaza se origina fuera de la organización
      - un grupo criminal tratando de robar información,
      - un huracán, yendo hacia el datacenter

https://www.sciencedirect.com/science/article/pii/S1877050914006528

Tipos de Amenazas

- Existe varios criterios para analizar los tipos de amenazas:
  - Según su agente amenazante:
    - Humanos: Una persona que en forma voluntaria o involuntaria genera el riesgo.
      - Hacker
      - Administrador del sistema (cometiendo un error)
    - Ambientales: Cualquier evento natural que pueda afectar a nuestro sistema o cambiar su comportamiento habitual.
      - Inundación
    - Tecnológicos: Cualquier evento relacionado con una afectación de la tecnología por su uso mismo.
      - Servidor que se le quema la fuente.

https://www.sciencedirect.com/science/article/pii/S1877050914006528

Tipos de Amenazas

- Existe varios criterios para analizar los tipos de amenazas:
  - Según la intención:
    - Malicioso: Cualquier forma de ataque con intención de afectar la confidencialidad, integridad o disponibilidad de un sistema.
    - No Malicioso: Afectación del sistema por un humano o un agente ambiental o tecnológico si la intención de causar daño.

https://www.sciencedirect.com/science/article/pii/S1877050914006528

Tipos de Amenazas

- Existe varios criterios para analizar los tipos de amenazas:
  - Según el impacto:
    - Destrucción de la información (disponibilidad)
    - Corrupción de la información (integridad)
    - Revelar información (confidencialidad)
    - Robo del servicio (sin afectar el uso de otros usuarios, fraude)
    - Denegación de servicio (disponibilidad)
    - Elevación de privilegios (bypass de ACL)
    - Uso ilegal (para lograr otros objetivos)

https://www.sciencedirect.com/science/article/pii/S1877050914006528

Tipos de Vulnerabilidades

- Es una debilidad en un activo o grupo de activos que permite que un atacante pueda aproveche para lograr un acceso que no tiene permitido.



https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- Deficiencias en el código fuente de las aplicaciones, las librerías y herramientas utilizadas para crear las aplicaciones de la organización.
- Usualmente son fallas en los controles o una lógica deficiente.



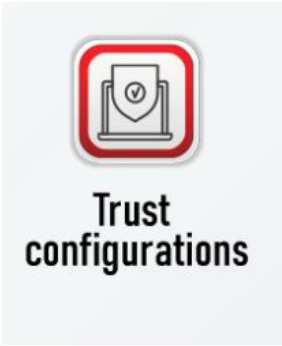
- Corresponde a sistemas configurados incorrectamente o con opciones de seguridad insuficientes.
- Es la forma más común de vulnerabilidad en los servicios de nube.



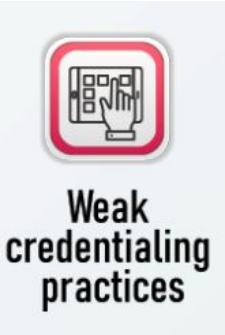
https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- Relaciones de confianza que no validen correctamente su uso, o no revaliden la autenticidad del sujeto.
- Ejemplo: NFS no valida al usuario antes de dar acceso a los archivos



- El uso de políticas de password inseguras o la exposición de credenciales que pueda facilitar a los atacantes acceder o adivinar las credenciales de autenticación
- La falta de MFA.



https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- El uso de algoritmos débiles de encripción que permitan robar las claves de encripción.



- Una persona que incumple las reglas o buenas prácticas con fines de cometer daño o fraude.



https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- Cualquier causa humana que sin intención genera un problema de seguridad.
- Toda forma de ingeniería social cae en esta categoría.
- Amenazas hacia los empleados para realizar tareas que comprometan la seguridad.



- Procedimiento inadecuados para el proceso de autenticación (no relacionado con las credenciales).
- Formas de reseteo de password. Uso de preguntas triviales o de información pública.
- Falta de un fail-safe.

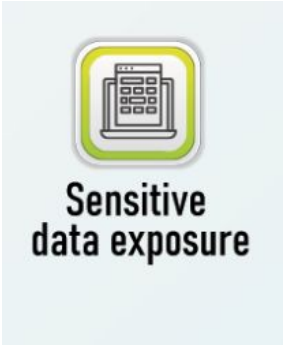


https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- Cualquier forma de inyección de código que permite a una aplicación realizar tareas distintas a las que fueron programadas.
- Generan puertas de entrada (backdoors) para ignorar controles de red o de autenticación.
- Facilita el acceso a los datos de la aplicación.
- Cualquier forma de exposición de datos confidenciales.
- Un error que muestra información interna o credenciales.
- Tachos de basura con documentos confidenciales o impresoras con documentos no reclamados





https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

Tipos de Vulnerabilidades

- Insuficiencia en la capacidad de monitorear los registros de actividad.
- Falta de visibilidad y capacidad de alertar actividad sospechosa.
- Generación de eventos sin procesos adecuados de análisis de estos logs.
- Cualquier recurso compartido entre dos organizaciones que no aisle lógicamente de manera correcta la información entre ellas.
- Un computadora compartida por dos usuarios que almacene información de cada uno en una carpeta compartida (/tmp)





https://www.spiceworks.com/it-security/vulnerability-management/articles/what-is-a-security-vulnerability/

MITRE Org

- El gobierno de EEUU creó hace tiempo una organización para centralizar la información relacionada con las vulnerabilidades de software.
- Esta organización cataloga la vulnerabilidad y permite tener una alerta temprana para los responsables y usuarios de ese software.
- CVE-YYYY-NNNNN es el código de todas las vulnerabilidades, siendo YYYY el año en que se descubrió y NNNNN un número de secuencia asignado.

CVEs

Year	2024	2023	2022	2021	2020	2019	2018	2017	2016
Qtr4	11,073	7,876	6,231	5,200	4,387	4,822	3,614	3,570	1,590
Qtr3	8,591	6,936	6,448	5,541	4,170	5,150	4,350	4,037	1,713
Qtr2	11,716	7,134	6,365	5,005	5,011	4,091	4,613	3,612	1,775
Qtr1	8,697	7,015	6,015	4,415	4,807	3,245	3,935	3,426	1,379
TOTAL	40,077	28,961	25,059	20,161	18,375	17,308	16,512	14,645	6,457



2026 (Hasta Oct)
75.242

2015	2014	2013	2012	2011	2010	2009	2008	2007	2006	2005
1,652	2,528	1,416	1,244	1,133	1,071	1,100	1,500	1,884	1,737	1,725
1,747	2,220	1,297	1,926	988	1,020	1,547	1,343	1,564	1,643	1,477
1,443	1,660	1,147	1,058	901	1,408	1,381	1,279	1,887	1,887	2,013
1,652	1,540	1,282	1,060	1,128	1,140	1,704	1,551	1,987	1,618	1,493
6,494	7,948	5,142	5,288	4,150	4,639	5,732	5,673	7,322	6,885	6,708

https://www.cve.org/About/Metrics

CVE-2021-44228: Log4j

CNA: Apache Software Foundation

Published: 2021-12-10 **Updated:** 2023-04-03
Title: Apache Log4j2 JNDI Features Do Not Protect Against Attacker Controlled LDAP And Other JNDI Related Endpoints

Description

Apache Log4j2 2.0-beta9 through 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1) JN
messages, and parameters
and other JNDI related endp
messages or log message pa
from LDAP servers when me
log4j 2.15.0, this behavior ha
2.16.0 (along with 2.12.2, 2.1
completely removed. Note th
does not affect log4net, log4cxx, or other Apache Logging Services projects.

CWE 3 Total

Learn more

- CWE-502: CWE-502 Deserialization of Untrusted Data
- CWE-400: CWE-400 Uncontrolled Resource Consumption
- CWE-20: CWE-20 Improper Input Validation

https://www.cve.org/CVERecord?id=CVE-2021-44228

CVE-2014-0160 - OpenSSL

CNA: Red Hat, Inc.

Published: 2014-04-07 **Updated:** 2022-11-15

Description

The (1) TLS and (2) DTLS implementations in OpenSSL 1.0.1 before 1.0.1g do not properly handle Heartbeat Extension packets, which allows remote attackers to obtain sensitive information from process memory via crafted packets that trigger a buffer over-read, as demonstrated by reading private keys, related to d1_both.c and t1_lib.c, aka the Heartbleed bug.

Product Status

Learn more

Information not provided

References

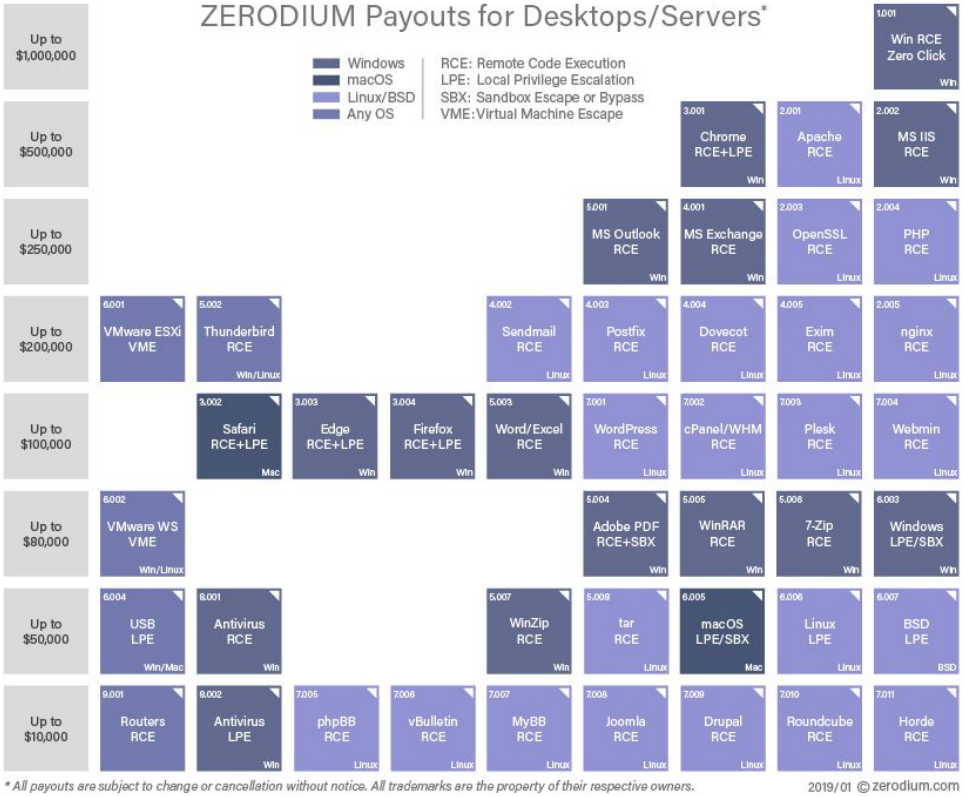
- https://support.f5.com/kb/en-us/solutions/public/15000/100/sol151

https://www.cve.org/CVERecord?id=CVE-2014-0160

Zero-day

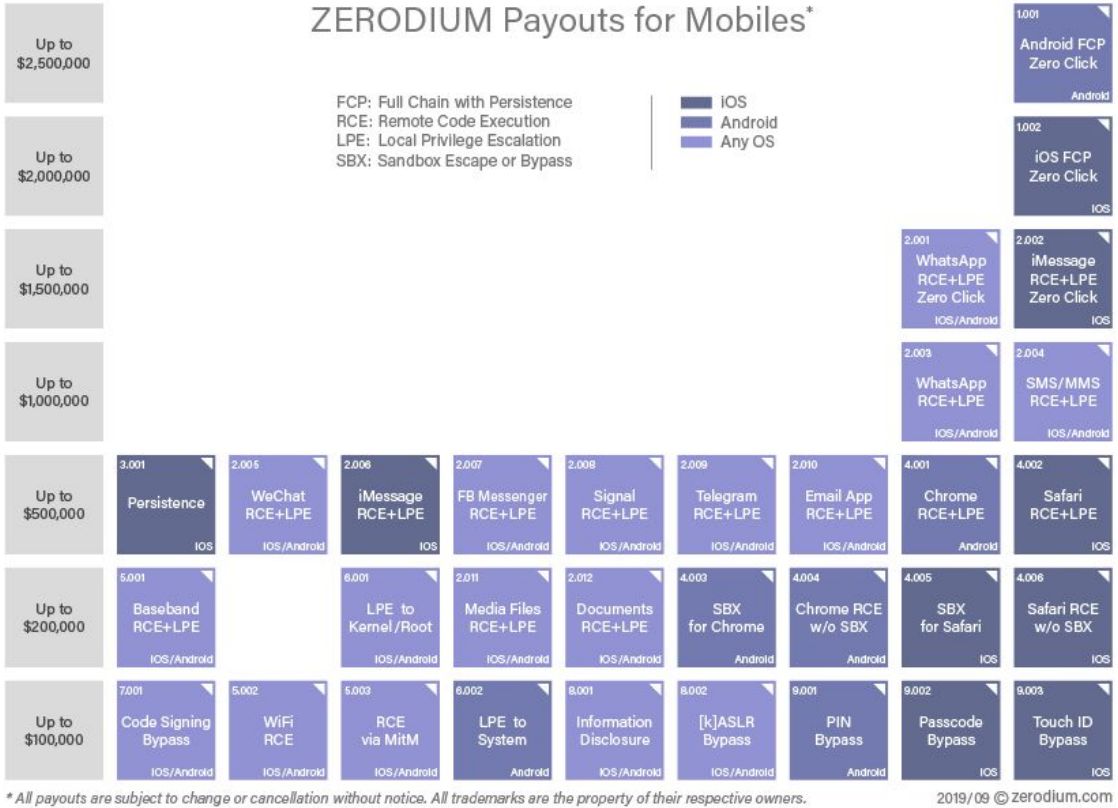
- Un exploit es el programa que permite usar una vulnerabilidad para generar una brech de seguridad.
- No todas las vulnerabilidades pueden ser *explotadas* fácilmente.
- Hasta que son reportadas y publicadas, estas vulnerabilidades se las conoce como zero-day.
- Los exploits zero-day tienen un alto valor en el mercado por que permite aprovechar vulnerabilidades para las cuales no hay prevención.

Zero-Day Payouts



https://zerodium.com/program.html

Zero-Day Payouts



https://zerodium.com/program.html

CWE - Common Weakness Enumeration

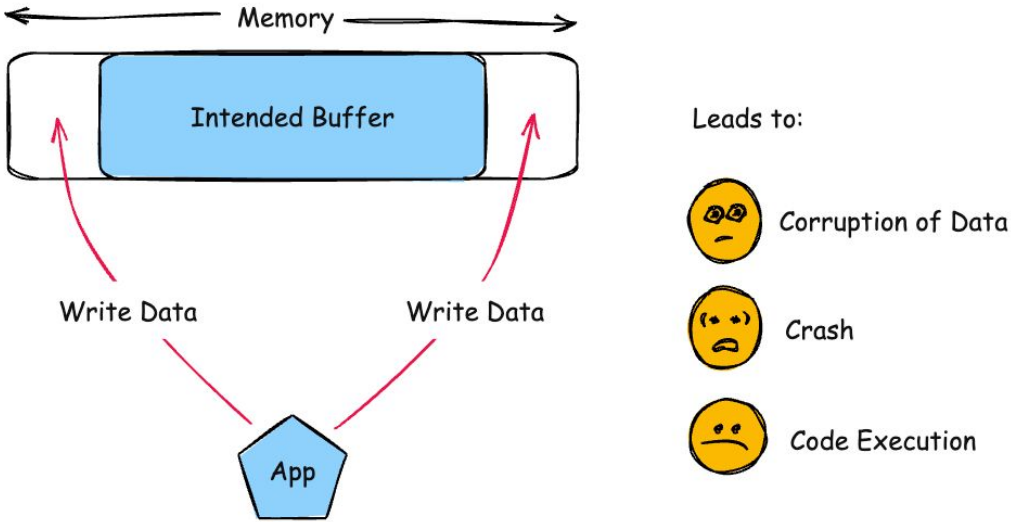
- CWE es un registro de las debilidades en el software que dan lugar a que las vulnerabilidades sucedan.
- Todos los años se publican la listas de las debilidades más comunes en la vulnerabilidades descubiertas.

1	Out-of-bounds Write	CWE-787 | CVEs in KEV: 70 | Rank Last Year: 1
2	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	CWE-79 | CVEs in KEV: 4 | Rank Last Year: 2
3	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	CWE-89 | CVEs in KEV: 6 | Rank Last Year: 3
4	Use After Free	CWE-416 | CVEs in KEV: 44 | Rank Last Year: 7 (up 3) ▲
5	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	CWE-78 | CVEs in KEV: 23 | Rank Last Year: 6 (up 1) ▲

https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html

CWE-787: Out-of-bounds Write

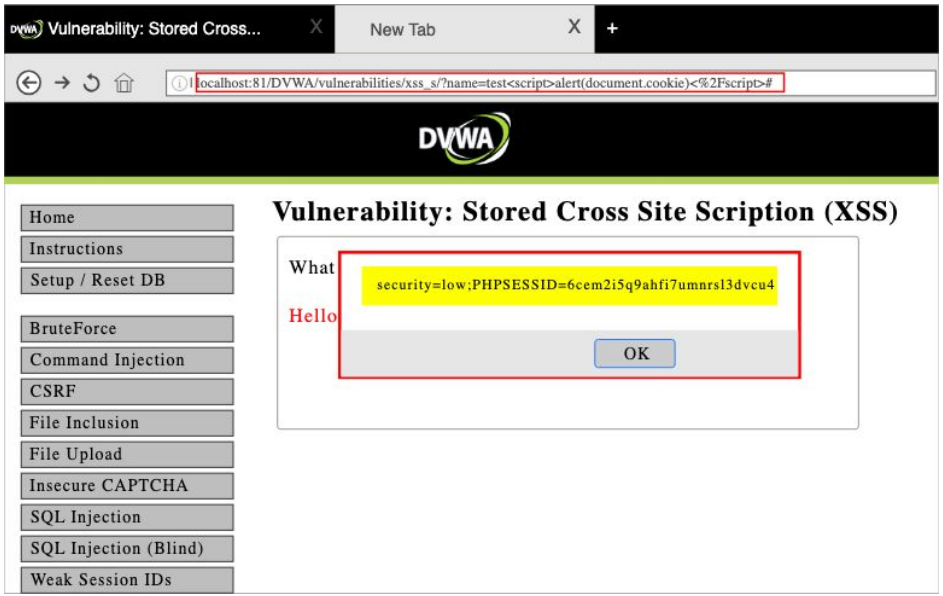
- Escritura en la memoria fuera de los límites originalmente definidos.
- Buffer overflow



https://cwe.mitre.org/data/definitions/787.html

CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

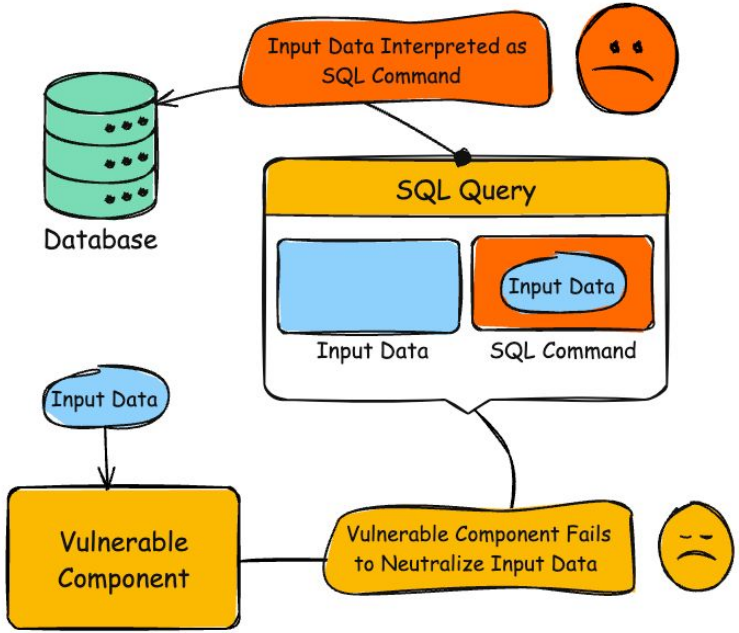
- Inyección de código malicioso en un link o requerimiento que termina siendo ejecutado por la víctima del ataque.



https://cwe.mitre.org/data/definitions/79.html
https://websitesecuritystore.com/blog/real-world-cross-site-scripting-examples/

CWE-89: AKA SQL Injection

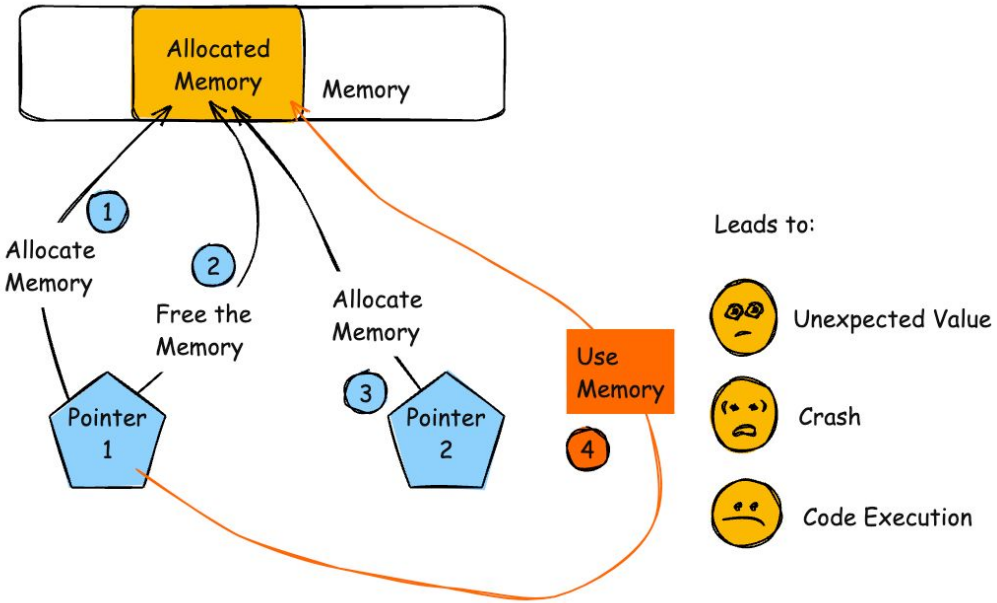
- Falta control en el ingreso de los usuarios que permite la ejecución de código SQL en las bases de datos



https://cwe.mitre.org/data/definitions/89.html

CWE-416: Use After Free

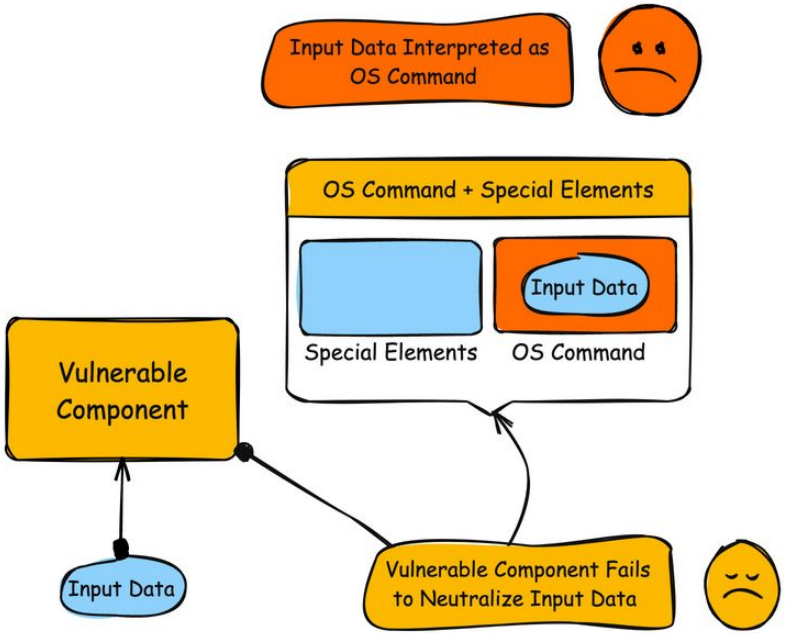
- Un recurso compartido que no es limpiado (wiped) entre usos de distintos usuarios.



https://cwe.mitre.org/data/definitions/416.html

CWE-78: OS Command/Shell Injection

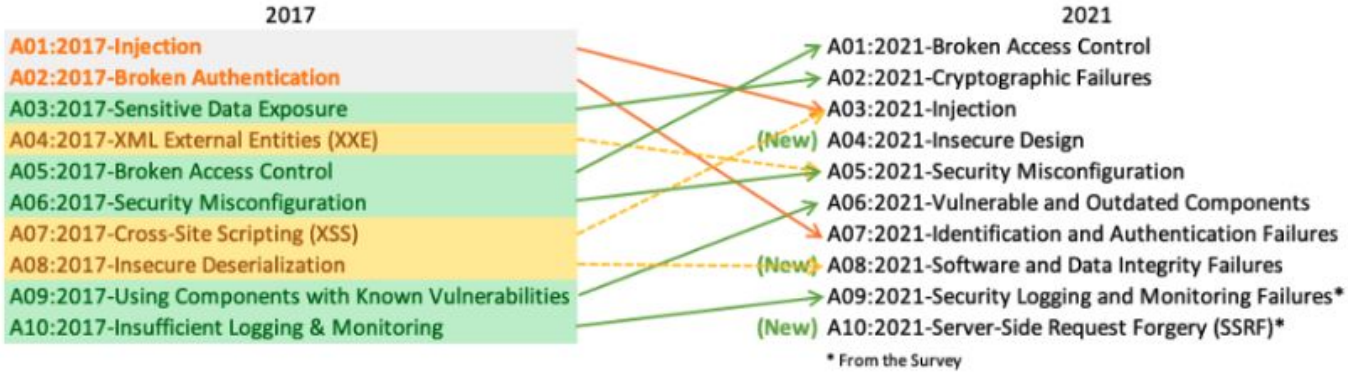
- Falta control en el ingreso de los usuarios que permite la ejecución de comandos en el sistema operativo.



https://cwe.mitre.org/data/definitions/78.html

OWASP Top 10

- OWASP es otra organización focalizada en combatir los problemas de seguridad en las aplicaciones Web.
- Regularmente publica las vulnerabilidades Web más comunes y las buenas prácticas para prevenirlas.



https://owasp.org/Top10/

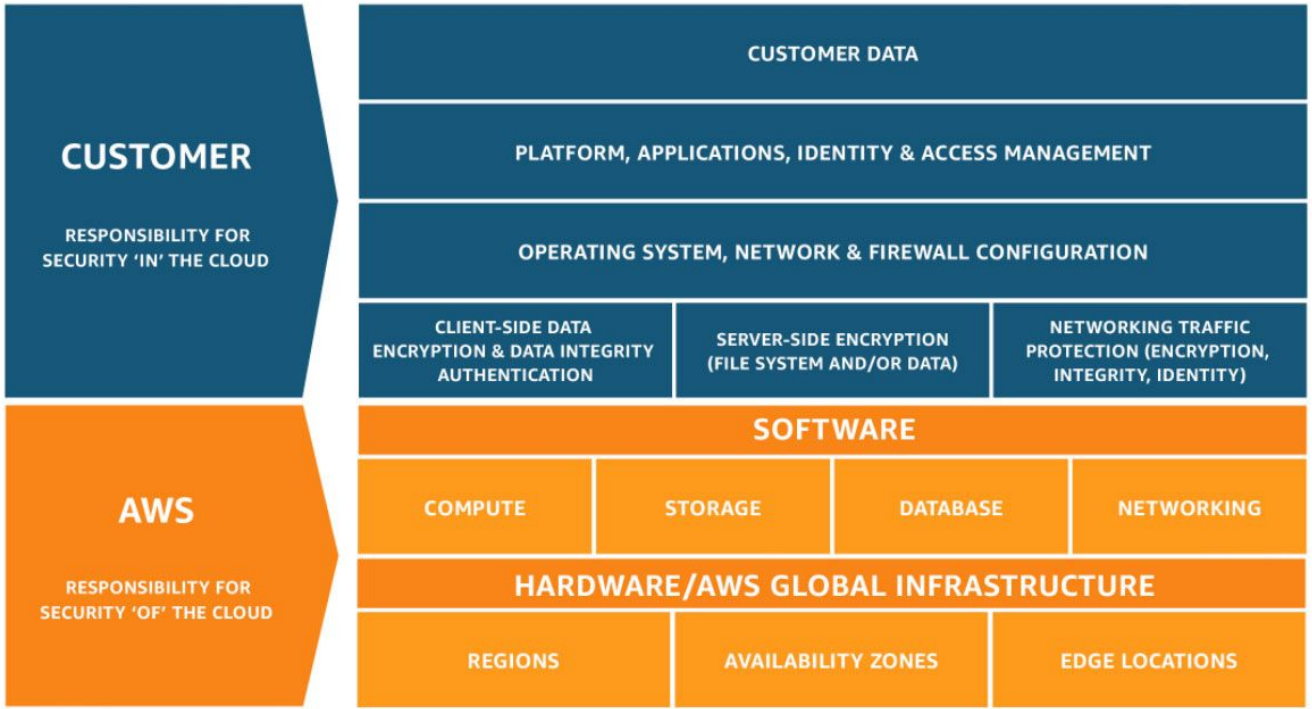
OWASP Cloud Top 10

- La "nube" es un conjunto de servicios externos, donde el proveedor se encarga de prevenir, evaluar o corregir vulnerabilidades, pero esto no es absoluto.
- Los principales problemas de seguridad en la nube viene de:
  - malas prácticas (malas configuraciones)
  - desconocimiento
  - problemas en lo que se despliega en la nube
  - uso de software con vulnerabilidades conocidas

https://owasp.org/www-project-cloud-native-application-security-top-10/

Cloud Vulnerabilities

- Modelo de responsabilidad proveedor/usuario



https://aws.amazon.com/compliance/shared-responsibility-model/

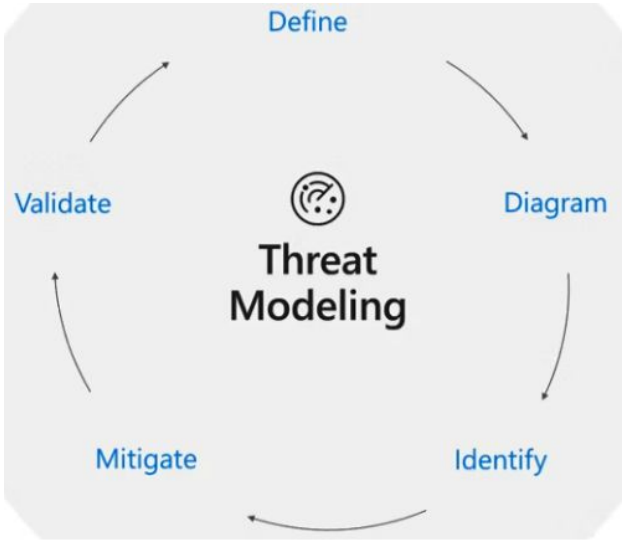


Criptografía y Seguridad

Threat Modeling

Threat modeling

- Es una técnica de ingeniería que se aplica para entender las amenazas, ataques, vulnerabilidades y contramedidas que pueden afectar a una aplicación.
- Se utiliza para mejorar el diseño y reducir el riesgo ante un ataque.



https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling

Modelo STRIDE

- El modelo STRIDE se utiliza para identificar las amenazas en un sistema.
- Fue desarrollado por Microsoft y es uno de los frameworks más utilizado para Threat Modeling.
- Utilizado en conjunto con un modelo de análisis de sistemas como los diagramas de flujo de datos (DFD) permiten identificar los riesgo y contramedidas que un sistema posee.

https://www.researchgate.net/publication/228793485_Experiences_Threat_Modeling_at_Microsoft
https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats

Modelo STRIDE

STRIDE THREAT MODEL

	Threat	Property Violated	Threat Definition
S	Spoofing	Authentication	Pretending to be something or someone other than yourself
T	Tampering	Integrity	Modifying something on disk, network, memory, or elsewhere.
R	Repudiation	Non- Repudiation	Claiming that you didn't do something or we're not responsible. Can be honest or false
I	Information Disclosure	Confidentiality	Providing information to someone not authorized to access it.
D	Denial of service	Availability	Exhausting resources needed to provide service.
E	Elevation of Privilege	Authorization	Allowing someone to do something they are not authorized to do.

https://www.linkedin.com/pulse/blog-50-enhance-your-cybersecurity-stride-threat-modeling-umang-mehta-szcff

Data Flow Model (DFD)

- Modelo para representar las partes de un sistema y los riesgos de cada una de estas.
- Analiza los elementos de cada categorías:
  - Procesos: contenedores, procesos ejecutables
  - Entidades externas al sistema: Personas, Otros sistemas
  - Flujos de datos: comunicaciones entre procesos
  - Almacenamientos de datos: Archivos, bases de datos, secretos

https://www.researchgate.net/publication/228793485_Experiences_Threat_Modeling_at_Microsoft

Data Flow Model (DFD)

- Para cada uno, se cruza con las amenazas definidas por STRIDE.
- Por cada riesgo identificado se debe definir una forma de gestionar ese riesgo: eliminarlo, mitigarlo, compensarlo o aceptarlo.

	Spoofing	Tampering	Repudiate	Info Disclose	Denial of Service	Elevate Privilege
External	x		x			
Process	x	x	x	x	x	x
Data Store		x	?	x	x	
Data Flow		x		x	x	

https://www.researchgate.net/publication/228793485_Experiences_Threat_Modeling_at_Microsoft

Threat modeling (OWASP)

- OWASP plantea un modelo similar al de Microsoft
  1. Definir el alcance del análisis, acotar el esfuerzo para que cubra lo suficiente de la aplicación pero no se exceda
  2. Determinar las amenazas
  3. Definir contramedidas y mitigación
  4. Revisar el trabajo realizado, los cambios y volver a empezar.

https://owasp.org/www-community/Threat_Modeling_Process

Threat modeling (Ejemplo OWASP)

- Scope

Threat Model Information (Sample)

Application Version: 1.0

Description: The college library website is the first implementation of a website to provide librarians and library patrons (students and college staff) with online services. As this is the first implementation of the website, the functionality will be limited. There will be three users of the application:

1. Students
2. Staff
3. Librarians

Staff and students will be able to log in and search for books, and staff members can request books. Librarians will be able to log in, add books, add users, and search for books.

Document Owner: David Lowry

Participants: David Rook

Reviewer: Eoin Keary

https://owasp.org/www-community/Threat_Modeling_Process

Lectura recomendada

Capítulo 12-13

Computer Security Art and Science
Matt Bishop

---

https://owasp.org/www-community/Threat_Modeling_Process

OWASP Threat Modeling